	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Carlos Chaves Mercado

Asignatura: Fundamentos de programación

Grupo: 8

No de Práctica(s): Práctica de estudio 08: Estructuras de selección

Integrante(s): Nolasco Ramírez Sofía

Díaz Rodríguez José Pablo

Cruz Gómez Lizette Jacqueline

Macay Martínez David

No. de lista o brigada: Mesa 6

Semestre: 2026-2

Fecha de entrega: 17 de abril de 2026

Observaciones:

CALIFICACIÓN: _____

Objetivo :

El alumno elaborará programas en C para la resolución de problemas básicos que incluyan las estructuras de repetición.

Introducción

Es de gran utilidad un sistema capaz de repetir una acción cuantas veces se requiera dentro de la ingeniería. En programación se implementan estructuras de control repetitivas que permiten controlar el flujo de datos basándose en condiciones lógicas establecidas por el programador, facilitando el procesamiento de series numéricas o tareas repetitivas.
Existen 4 mecanismos esenciales que se utilizaron en la práctica y serán mencionados a continuación:

- Sentencia *while*: Evalúa una condición antes de ejecutar cualquier instrucción. Si la condición es falsa desde el principio, el cuerpo del ciclo nunca se ejecutará. Por otro lado, si es verdadera se ejecutará el bloque del código y regresará al inicio para evaluar de nuevo.
- Sentencia *do while*: Este ciclo garantiza que las instrucciones se ejecuten al menos una vez, ya que la condición se evalúa al final. Si la condición es verdadera, repite el proceso.
- Estructura *for*: Su utilidad es ejecutar el bloque de código un número exacto de iteraciones. Agrupa 3 elementos en una sola línea:
  - Inicialización: Se define el punto de partida
  - Condición: El límite del ciclo o que cantidad de veces se ejecutará el código.
  - Actualización: Como cambia el contador en cada vuelta.

Desarrollo

La práctica 8 se desarrolló mediante la elaboración de un programa en lenguaje C en el cual se integraron conceptos trabajados previamente en prácticas anteriores y estructuras de control repetitivas. A continuación, se presentarán ejemplos de distintos programas realizados usando diferentes estructuras de repetición, para distintos problemas.

Programa1.c

Este programa genera la tabla de multiplicar de un número dado. básicos8ro se lee desde la entrada estándar (teclado).

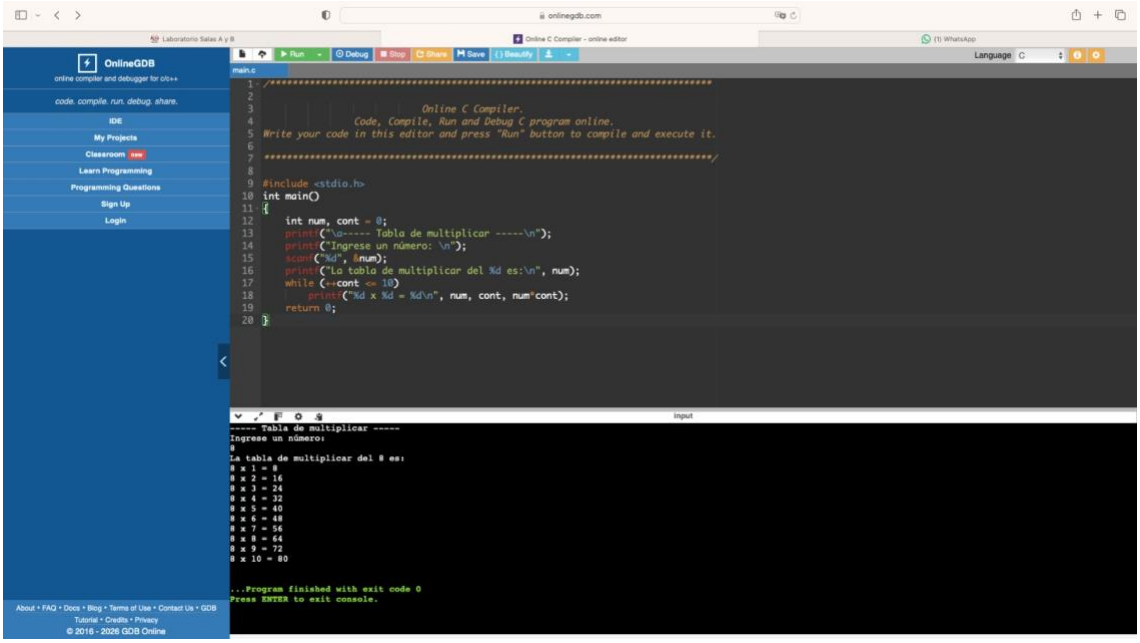

Figura 1. Código fuente del programa 1 en lenguaje C

Programa2.c

Este programa genera un ciclo infinito.

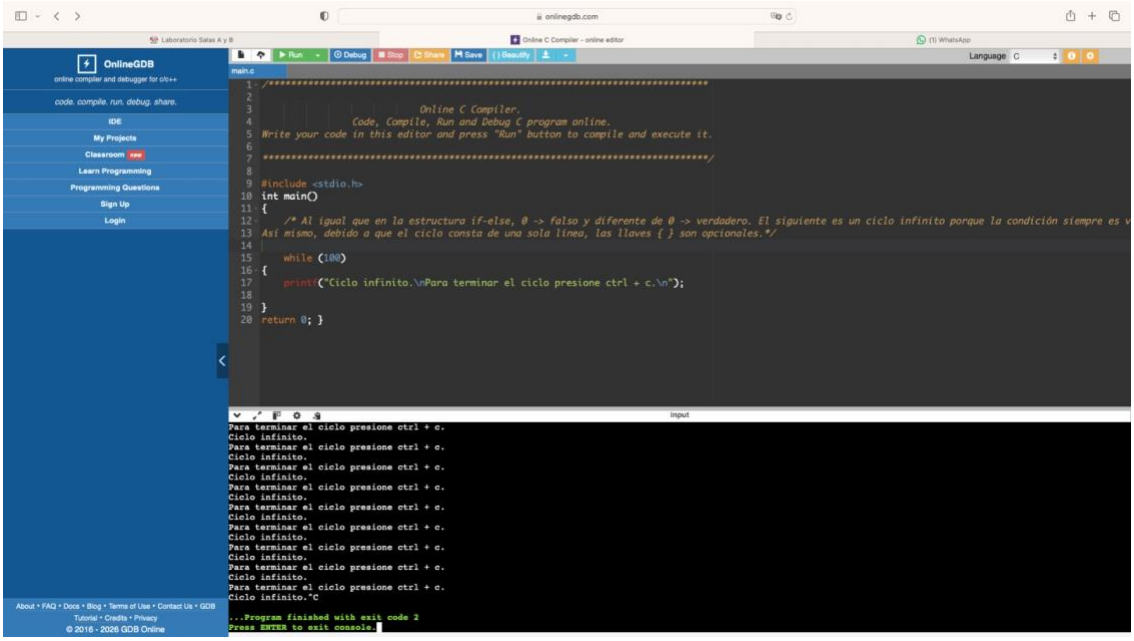

Figura 2. Código fuente del programa 2 en lenguaje C

Programa3.c

Este programa obtiene el promedio de calificaciones ingresadas por el usuario. Las calificaciones se leen desde la entrada estándar (teclado). La inserción de calificaciones termina cuando el usuario presiona una tecla diferente de ‘S’ o ‘s’.

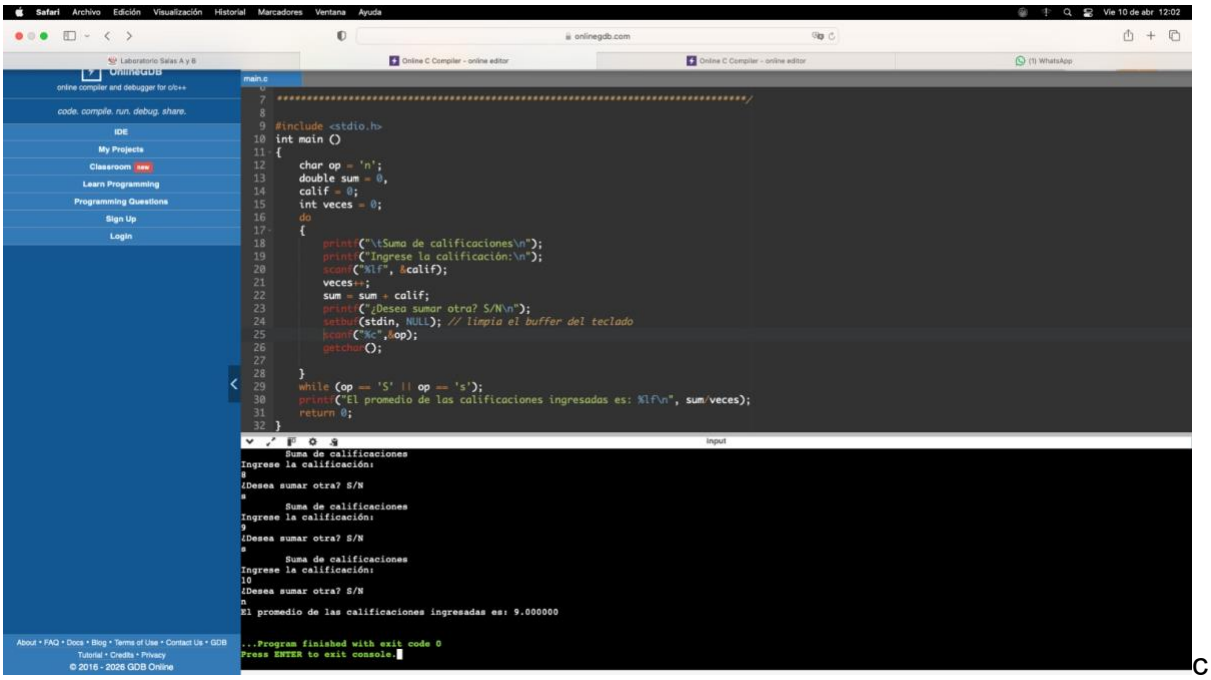

Figura 3. Código fuente del programa 3 en lenguaje C

Programa4.c

Este programa genera una calculadora básica

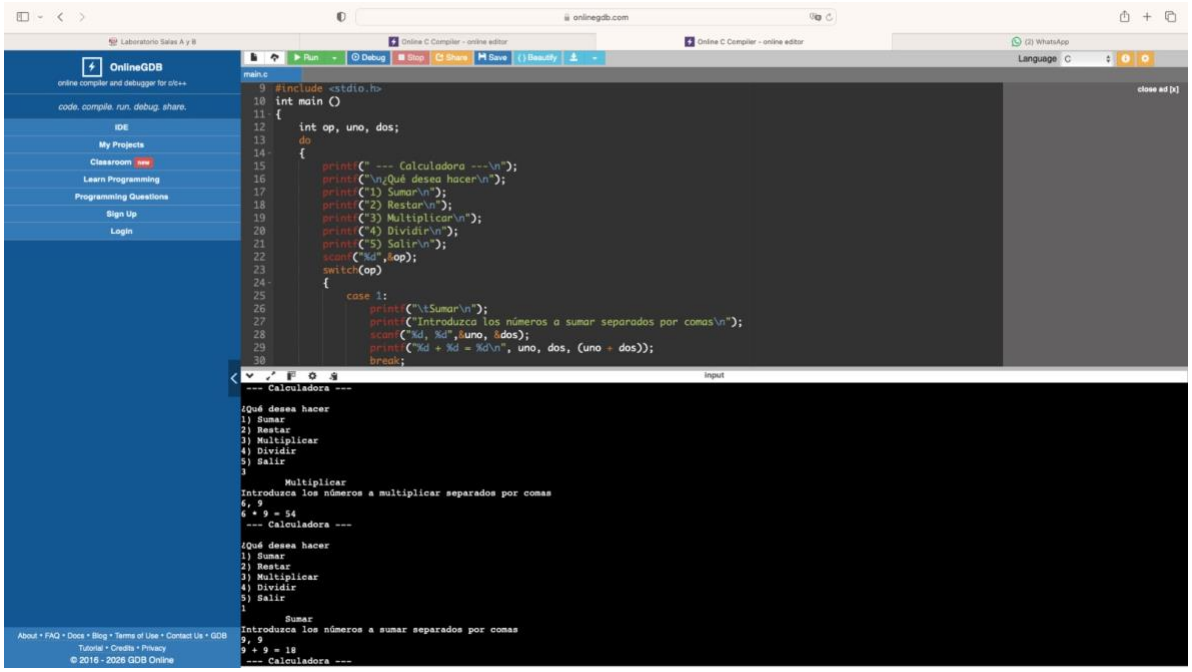

Figura 4.0 Código fuente del programa 4 en lenguaje C

```
31              case 2:
32                  printf("\tRestar\n");
33                  printf("Introduzca los números a restar separados por comas\n");
34                  scanf("%d, %d",&uno, &dos);
35                  printf("%d - %d = %d\n", uno, dos, (uno - dos));
36                  break;
37              case 3:
38                  printf("\tMultiplicar\n");
39                  printf("Introduzca los números a multiplicar separados por comas\n");
40                  scanf("%d, %d",&uno, &dos);
41                  printf("%d * %d = %d\n", uno, dos, (uno * dos));
42                  break;
43              case 4:
44                  printf("\tDividir\n");
45                  printf("Introduzca los números a dividir separados por comas\n");
46                  scanf("%d, %d",&uno, &dos);
47                  printf("%d / %d = %.2lf\n", uno, dos, ((double)uno / dos));
48                  break;
49
50          }
51
52      }
```

```
6 * 9 = 54
 --- Calculadora ---

¿Qué desea hacer
1) Sumar
2) Restar
3) Multiplicar
4) Dividir
5) Salir
1
        Sumar
Introduzca los números a sumar separados por comas
9, 9
9 + 9 = 18
 --- Calculadora ---

¿Qué desea hacer
1) Sumar
2) Restar
3) Multiplicar
4) Dividir
5) Salir
5

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 4.1 Código fuente del programa 4 en lenguaje C

```
34              scanf("%d, %d",&uno, &dos);
35              printf("%d - %d = %d\n", uno, dos, (uno - dos));
36              break;
37          case 3:
38              printf("\tMultiplicar\n");
39              printf("Introduzca los números a multiplicar separados por comas\n");
40              scanf("%d, %d",&uno, &dos);
41              printf("%d * %d = %d\n", uno, dos, (uno * dos));
42              break;
43          case 4:
44              printf("\tDividir\n");
45              printf("Introduzca los números a dividir separados por comas\n");
46              scanf("%d, %d",&uno, &dos);
47              printf("%d / %d = %.2lf\n", uno, dos, ((double)uno / dos));
48              break;
49
50      }
51
52  }
53  while (op != 5);
54
55 }
```

```
6 * 9 = 54
 --- Calculadora ---

¿Qué desea hacer
1) Sumar
2) Restar
3) Multiplicar
4) Dividir
5) Salir
1
        Sumar
Introduzca los números a sumar separados por comas
9, 9
9 + 9 = 18
 --- Calculadora ---

¿Qué desea hacer
1) Sumar
2) Restar
3) Multiplicar
4) Dividir
5) Salir
5

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 4.2 Código fuente del programa 4 en lenguaje C

Programa5.c

Este programa genera un arreglo unidimensional de 5 elementos y se accede a cada elemento del arreglo a través de un ciclo for.

```c
/*****************************************************************************/

#include <stdio.h>
int main ()
{
    int enteroNumAlumnos = 5;
    float realCalif = 0.0, realPromedio = 0.0;
    printf("\tPromedio de calificaciones\n");
    for (int indice = 0 ; indice < enteroNumAlumnos ; indice++)
    {
        printf("\nIngrese la calificación del alumn %d\n", indice+1);
        scanf("%f",&realCalif);
        realPromedio += realCalif;

    }
    printf("\nEl promedio de las calificaciones ingresadas es: %f\n",
    realPromedio/enteroNumAlumnos);
    return 0;

}
```

```
Promedio de calificaciones

Ingrese la calificación del alumn 1
9

Ingrese la calificación del alumn 2
8

Ingrese la calificación del alumn 3
10

Ingrese la calificación del alumn 4
6

Ingrese la calificación del alumn 5
9

El promedio de las calificaciones ingresadas es: 8.400000


...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 5. Código fuente del programa 5 en lenguaje C

Programa6.c

Este programa define un valor por defecto para el tamaño del arreglo de tal manera que si el tamaño de éste cambia, solo se debe modificar el valor de la constante MAX.

```c
                    Online C Compiler.
            Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

/*****************************************************************************/

#include <stdio.h>
#define MAX 5
int main ()
{
    int arreglo[MAX], cont;
    for (cont=0; cont<MAX; cont++)
    {
        printf("Ingrese el valor %d del arreglo: ", cont+1);
        scanf("%i", &arreglo[cont]);
    }
    printf("El valor ingresado para cada elemento del arreglo es:\n[");
    for (cont=0; cont<MAX; cont++)
    {
        printf("%d\t", arreglo[cont]);

    }
    printf("]\n");
    return 0;
}
```

```
Ingrese el valor 1 del arreglo: 600
Ingrese el valor 2 del arreglo: 390
Ingrese el valor 3 del arreglo: 800
Ingrese el valor 4 del arreglo: 100
Ingrese el valor 5 del arreglo: 500
El valor ingresado para cada elemento del arreglo es:
[600    390     800     100     500     ]


...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 6. Código fuente del programa 6 en lenguaje C

Ejemplo 1:

a) *Se requiere un algoritmo para obtener la edad promedio de un grupo de N alumnos. Realice el diagrama de flujo, el pseudocódigo y el diagrama N / S para representarlo, utilizando los 3 tipos de estructuras de ciclo.*

Nombre de la variable	Descripción	Tipo
C	Contador	Entero
ED	Edad de cada alumno	Entero
SU	Suma de las edades	Entero
NU	Número de alumnos	Entero
PR	Edad promedio	Real

El presente ejercicio tiene como objetivo desarrollar un algoritmo capaz de calcular la edad promedio de un grupo de alumnos, a partir de la captura de sus edades individuales. Para lograrlo, se emplean estructuras repetitivas, las cuales permiten ejecutar un conjunto de instrucciones varias veces hasta cumplir una condición determinada.

En primer lugar, se declara una variable que representa el número total de alumnos (NU), la cual es proporcionada por el usuario al inicio del programa. Esta variable es fundamental, ya que determina cuántas veces se repetirá el proceso de captura de datos.

Posteriormente, se inicializan dos variables auxiliares:

- Un acumulador (SU), que se encarga de almacenar la suma total de las edades ingresadas.
- Un contador (C), que permite llevar el control del número de alumnos procesados durante la ejecución del ciclo.

Una vez inicializadas las variables, el algoritmo entra en una estructura repetitiva, dentro de la cual se realizan las siguientes acciones en cada iteración:

1. Se solicita al usuario la edad de un alumno (ED).
2. Se añade el valor ingresado al acumulador (SU = SU + ED).
3. Se incrementa el contador en una unidad (C = C + 1), con el fin de acercarse al número total de alumnos definidos inicialmente.

Este proceso se repite hasta que el contador alcanza el valor de NU, lo que indica que ya se han ingresado todas las edades requeridas.

Al finalizar el ciclo, se procede a calcular el promedio de edades (PR), utilizando la fórmula:

Finalmente, el resultado obtenido se muestra en pantalla, permitiendo al usuario conocer la edad promedio del grupo.

```
/*****************************************************************************

                    Online C Compiler.
            Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

*****************************************************************************/

#include <stdio.h>

int main()
{
    int NU, ED;
    int C = 1;
    int SU = 0;
    float PR;
    printf("Ingrese el número de alumnos:");
    scanf("%d", &NU);
    while (C<=NU)
    {
        printf("Ingrese la edad del alumno %d: ", C);
        scanf("%d", &ED);
        SU = SU + ED;
        C = C + 1 ;
    }
       if (NU > 0)
    {
     PR = (float)SU / NU;
     printf ("\nLa edad promedio del grupo es :%.2f\n",PR);
    }
    else
    {
        printf ("\nNo se ingresaron alumnos.\n");
    }
    return 0;
}
```

Figura 7. Código fuente del ejemplo 1 en lenguaje C, estructura *while*

```
Write your code in this editor and press "Run" button to compile and execute it.

*****************************************************************************/

#include <stdio.h>

int main()
{
    int NU, C = 1, ED;
    int SU = 0;
    float PR;

    printf("Ingrese el numero de alumnos: ");
    scanf("%d", &NU);

    do
    {
        printf("Edad del alumno %d: ", C);
        scanf("%d", &ED);

        SU = SU + ED;
        C++;
    } while (C <= NU);

    PR = (float)SU / NU;

    printf("\nEdad promedio: %.2f\n", PR);

    return 0;
}
```

```
Ingrese el numero de alumnos: 5
Edad del alumno 1: 5
Edad del alumno 2: 6
Edad del alumno 3: 4
Edad del alumno 4: 5
Edad del alumno 5: 3

Edad promedio: 4.60

...Program finished with exit code 0
Press ENTER to exit console.
```

Figura 8. Código fuente del ejemplo 1 en lenguaje C, estructura *do-while*

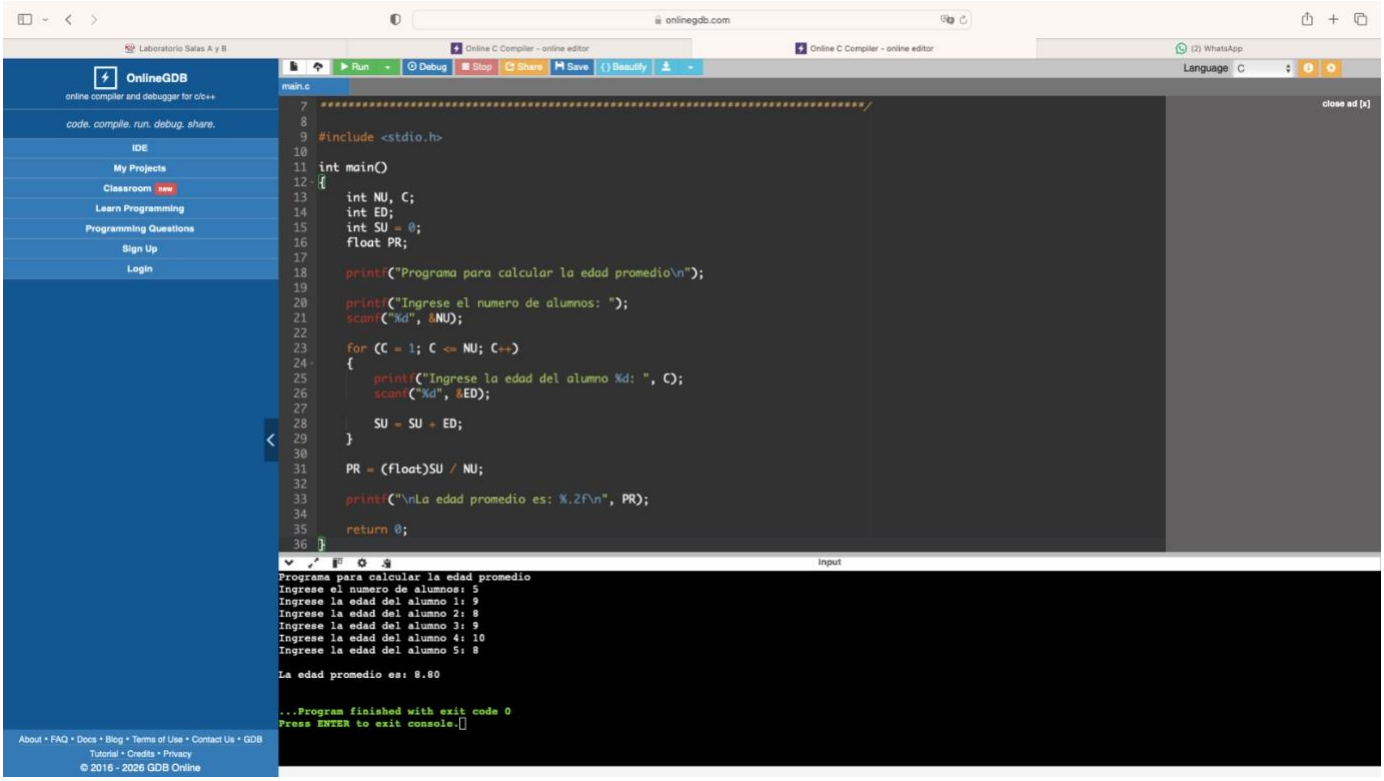

Figura 9. Código fuente del ejemplo 1 en lenguaje C, estructura *for*

Análisis de resultados:

Programa1.c

El uso de una estructura lógica repetitiva, donde una variable de tipo entero num es multiplicada del 1 al 10, permiten visualizar todos los números que integran la tabla de multiplicación sin necesidad de hacer cálculo mental.

Programa2.c

Un bucle de tipo infinito permite que la condición de salida nunca sea falsa de forma natural, manteniendo el programa en ejecución hasta que un evento externo o una sentencia de ruptura lo detenga.

Programa3.c

Se presenta la estructura inicial del algoritmo en código, donde se definen las variables necesarias para resolver el problema. Entre ellas destacan las variables para almacenar los datos de entrada, como la calificación (por ejemplo, **calif**), una variable acumuladora (**sum**) y una variable para el resultado final.

Esta figura trata del **planteamiento inicial del programa**, ya que muestra cómo se comienza a construir la lógica del problema: se solicita un dato al usuario, se almacena en una variable y se realiza una operación básica, generalmente sumarlo en el acumulador.

Sin embargo, en esta etapa el código aún no incluye una estructura repetitiva, por lo que el programa solo procesa un valor a la vez. Esto indica que el algoritmo todavía está incompleto, ya que falta implementar un ciclo que permita repetir el proceso automáticamente para múltiples datos.

Programa4.c

se observa un avance importante en el desarrollo, donde ya se implementa una estructura repetitiva (**do-while**) que permite ejecutar el menú de opciones varias veces sin tener que reiniciar el programa manualmente.

En el código se definen variables como `op` (para elegir la opción del menú) y `uno`, `dos` (de tipo flotante para almacenar los números que se van a operar). Además, se utiliza la estructura **switch**, la cual permite seleccionar de forma organizada la operación a realizar (sumar, restar, multiplicar o dividir) dependiendo de la opción ingresada por el usuario.

Este programa funciona como una calculadora interactiva. A diferencia de los programas anteriores de la práctica, aquí el ciclo garantiza que el usuario pueda realizar múltiples cálculos en una sola sesión, lo que lo hace mucho más completo.

Sin embargo, aunque ya existe repetición, el control del ciclo depende totalmente de que el usuario elija la opción "Salir", por lo que el manejo de la condición de parada es el aprendizaje clave en esta figura.

Programa5.c

El programa está desarrollado en lenguaje C y tiene como objetivo calcular el promedio de calificaciones de un grupo de alumnos. Para ello, comienza incluyendo la librería estándar `stdio.h`, la cual permite utilizar funciones de entrada y salida como `printf` y `scanf`.

Dentro de la función principal `main`, se declaran tres variables: una de tipo entero llamada `enteroNumAlumnos`, que almacena el número de alumnos (en este caso, 5), y dos variables de tipo flotante (`realCalif` y `realPromedio`). La variable `realCalif` se utiliza para guardar cada calificación que el usuario introduce, mientras que `realPromedio` sirve para acumular la suma de todas las calificaciones.

El programa muestra en pantalla un mensaje inicial indicando que se calculará el promedio de calificaciones. Posteriormente, utiliza una estructura repetitiva `for`, la cual se ejecuta cinco veces, una por cada alumno. En cada iteración, se solicita al usuario que ingrese la calificación correspondiente, se captura el valor mediante `scanf` y se suma a la variable acumuladora `realPromedio`.

Una vez finalizado el ciclo, el programa calcula el promedio dividiendo la suma total de las calificaciones entre el número de alumnos. Finalmente, muestra en pantalla el resultado del promedio y termina su ejecución retornando un valor de 0.

Programa6.c

El programa está escrito en lenguaje C y tiene como objetivo capturar y mostrar los valores de un arreglo de tamaño fijo. Al inicio, se incluye la librería `stdio.h`, necesaria para realizar operaciones de entrada y salida. Además, se define una constante llamada `MAX` con valor 5, la cual representa el tamaño del arreglo que se utilizará en el programa.

Dentro de la función principal `main`, se declaran dos variables: un arreglo de tipo entero llamado `arreglo` con tamaño `MAX`, y una variable entera `cont` que funciona como contador en los ciclos. El programa utiliza una estructura repetitiva `for` para solicitar al usuario que ingrese los valores del arreglo. Este ciclo se ejecuta desde 0 hasta `MAX-1`, permitiendo almacenar cada número ingresado en la posición correspondiente del arreglo mediante la función `scanf`.

Una vez que todos los valores han sido ingresados, el programa muestra un mensaje indicando que se imprimirán los elementos del arreglo. Posteriormente, se utiliza un segundo ciclo `for` para recorrer nuevamente el arreglo y mostrar cada uno de sus elementos en pantalla con la función `printf`. Los valores se presentan de forma ordenada, simulando la estructura de un arreglo entre corchetes.

Finalmente, el programa imprime un corchete de cierre, termina su ejecución con `return 0` y concluye correctamente. En resumen, este código ejemplifica el uso de arreglos, ciclos repetitivos y entrada/salida de datos para almacenar y mostrar información ingresada por el usuario.

Ejercicio 1.

Aunque el problema resuelto es el mismo en las tres figuras, la diferencia principal radica en el tipo de estructura repetitiva utilizada en cada caso:

En primer lugar, en la **Figura 7**, se emplea la estructura **while**, la cual se caracteriza por evaluar la condición antes de ejecutar el bloque de instrucciones. Esto significa que el ciclo solo se llevará a cabo si la condición es verdadera desde el inicio. En este caso, la condición generalmente está relacionada con que el contador sea menor o igual al número total de alumnos (NU).

Una ventaja importante de esta estructura es que evita ejecuciones innecesarias cuando la condición inicial no se cumple. Sin embargo, también implica que, si el valor de entrada no es válido (por ejemplo, si NU es igual a cero o negativo), el ciclo no se ejecutará en absoluto, lo que podría provocar que no se capture ningún dato. Por ello, el uso de **while** es más adecuado en situaciones donde no se garantiza que el proceso deba realizarse al menos una vez.

Por otro lado, en la **Figura 8**, se utiliza la estructura **do-while**, la cual presenta una diferencia fundamental respecto al while: la condición se evalúa después de ejecutar el bloque de instrucciones. Esto asegura que el ciclo se ejecute al menos una vez, independientemente de si la condición se cumple o no desde el inicio.

Esta característica resulta especialmente útil en programas donde es indispensable realizar al menos una acción, como la captura inicial de datos. No obstante, también puede representar un inconveniente

si los datos iniciales no son válidos, ya que el programa ejecutará una iteración aunque no debería hacerlo. En este sentido, el uso de **do-while** implica asumir que al menos una ejecución del proceso es necesaria.

Finalmente, en la **Figura 9**, se emplea la estructura **for**, la cual integra en una sola línea la inicialización del contador, la condición de continuación y la actualización del mismo. Esta estructura es especialmente útil cuando se conoce de antemano el número exacto de iteraciones que se deben realizar, como ocurre en este ejercicio, donde el número de alumnos (NU) se define desde el inicio.

El uso de **for** permite una mejor organización y claridad en el código, ya que reduce la posibilidad de errores relacionados con la inicialización o actualización del contador. Además, facilita la lectura del algoritmo, haciendo evidente cuántas veces se repetirá el ciclo. Sin embargo, puede ser menos flexible que las otras estructuras en situaciones donde el número de repeticiones no está definido previamente.

Conclusión

Diaz Rodríguez José Pablo:

Nolasco Ramírez Sofia: Durante el desarrollo de esta práctica se logró comprender y aplicar de manera efectiva el uso de las estructuras de repetición en el lenguaje de programación C, tales como while, do-while y for. A través de los distintos programas elaborados, se pudo observar cómo estas estructuras permiten automatizar procesos, facilitar la manipulación de datos y resolver problemas de forma más eficiente.

Asimismo, se reforzaron conceptos fundamentales como el uso de variables, acumuladores, contadores, arreglos y la entrada y salida de datos. Cada programa representó un caso práctico diferente, desde la generación de tablas, ciclos infinitos y calculadoras, hasta el manejo de arreglos y el cálculo de promedios, lo que permitió visualizar la versatilidad de las estructuras repetitivas en distintos contextos.

De igual manera, se identificaron las diferencias y aplicaciones específicas de cada tipo de ciclo, comprendiendo cuándo es más conveniente utilizar cada uno según el problema a resolver. Esto contribuye a desarrollar una lógica de programación más clara y estructurada.

Cruz Gómez Lizette Jacqueline: Durante el desarrollo de la práctica se logró comprender y aplicar de manera efectiva el uso de las estructuras de repetición en el lenguaje C, como while, do-while y for. A través de la elaboración de distintos programas, se observó cómo estos ciclos permiten automatizar procesos, optimizar el flujo del programa y facilitar la resolución de problemas que requieren la ejecución repetitiva de instrucciones.Asimismo, se reforzaron conceptos fundamentales como el uso de variables, contadores, acumuladores y arreglos, así como la interacción con el usuario mediante entrada y salida de datos. En conjunto, la práctica permitió fortalecer la lógica de programación y

desarrollar un pensamiento más estructurado, comprendiendo cuándo y cómo utilizar cada tipo de ciclo de manera adecuada.

Macay Martínez David: A través de esta práctica, se logró la implementación exitosa de programas en lenguaje C orientados a la resolución de problemas básicos. Mediante el uso de estructuras de repetición como for, while y do-while, se validó la capacidad de automatizar procesos iterativos, optimizando el flujo de control y reduciendo la redundancia en el código fuente.

Bibliografía

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.